==========

==========

CHAPTER 9 — MEMORY BARRIERS (DSB, DMB, ISB)

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of ARMv8 memory barriers, ordering rules, pipeline effects, page-table update requirements, and hypervisor interactions.

****

****

9.1 — Introduction to Memory Barriers

****

****

Memory barriers (also called *fences*) enforce ordering constraints on:

- memory accesses
- page table updates
- TLB invalidations

- instruction pipeline behavior
- device memory operations
- cache maintenance operations

ARMv8 provides three primary barrier instructions:

- **DSB** — Data Synchronization Barrier
- **DMB** — Data Memory Barrier
- **ISB** — Instruction Synchronization Barrier

These instructions ensure correctness in multi-core, multi-level translation environments.

****

****

9.2 — Why Barriers Are Required

****

****

Without barriers:

- page table updates may not be visible
- TLB invalidations may not complete
- speculative execution may use stale mappings
- device memory operations may reorder
- SMP cores may observe inconsistent memory state
- hypervisor may see inconsistent Stage-2 mappings

Teaching Note

Barriers prevent **silent corruption**, the most dangerous class of MMU bugs.

****

****

9.3 — DSB (Data Synchronization Barrier)

****

****

Definition

DSB ensures that **all memory accesses** before the barrier are **complete and globally visible** before any instruction after the barrier executes.

Effects

- completes all outstanding memory operations
- flushes store buffers
- ensures page table writes reach memory
- ensures TLB invalidations complete
- synchronizes device memory operations

Syntax

```
DSB  SY
```

Use Cases

- page table updates
- TLBI sequences
- device register writes
- DMA buffer synchronization
- hypervisor memory remapping

Implementation Note

DSB is the strongest barrier.

****

****

9.4 — DMB (Data Memory Barrier)

****

****

Definition

DMB ensures **ordering** of memory accesses but does not guarantee completion.

Effects

- enforces ordering between loads/stores
- prevents reordering across the barrier
- ensures memory visibility ordering

Syntax

```
DMB SY
```

Use Cases

- lock-free algorithms
- SMP synchronization
- ordering device memory accesses

Teaching Note

DMB is weaker than DSB.

****

****

9.5 — ISB (Instruction Synchronization Barrier)

****

****

Definition

ISB flushes the instruction pipeline and ensures that subsequent instructions execute with updated architectural state.

Effects

- flushes pipeline
- ensures updated TTBR/TCR/MAIR are visible
- ensures updated SCTLR/WXN/PAN are visible
- ensures updated exception vectors are visible

Syntax

```
ISB
```

Use Cases

- after TTBR updates
- after enabling/disabling MMU
- after changing SCTLR_EL1
- after changing translation regime

Implementation Note

ISB does not affect memory ordering.

****

****

9.6 — Barrier Strength Comparison

****

****

Barrier	Completes Memory Ops	Orders Memory Ops	Flushes Pipeline
DSB	Yes	Yes	No
DMB	No	Yes	No
ISB	No	No	Yes

Teaching Note

DSB is required for page table updates. ISB is required after changing translation registers.

****

****

9.7 — Barriers and Page Table Updates

****

****

Correct sequence for updating page tables:

1. Write descriptor
2. **DSB SY** — ensure descriptor visible
3. **TLBI** — invalidate stale TLB entries
4. **DSB SY** — ensure TLBI complete
5. **ISB** — flush pipeline

Implementation Note

Skipping any step can corrupt TLB state.

****

****

9.8 — Barriers and Break-Before-Make (BBM)

****

****

BBM requires:

1. Invalidate old descriptor
2. **DSB SY**
3. Install new descriptor
4. **DSB SY**
5. **ISB**

Teaching Note

BBM prevents aliasing between old and new mappings.

****

****

9.9 — Barriers and TLB Invalidation

****

****

TLBI instructions must be followed by:

```
DSB SY
ISB
```

Why?

- DSB ensures invalidation completes
- ISB ensures pipeline sees new translations

Implementation Note

TLBI without barriers is unsafe.

****

****

9.10 — Barriers and Device Memory

****

****

Device memory is:

- strongly ordered
- non-cacheable
- non-speculative

Barriers ensure:

- correct ordering of MMIO writes
- correct ordering of MMIO reads
- correct ordering of DMA operations

Example

```
STR x0, [MMIO_REG]
DSB SY
```

****

****

9.11 — Barriers and DMA

****

****

DMA engines operate outside CPU caches.

Barriers ensure:

- CPU writes reach memory before DMA reads
- DMA writes reach memory before CPU reads
- cache maintenance operations complete

Example

```
DC CVAC, x0
DSB SY
```

****

****

9.12 — Barriers and SMP Systems

****

****

Multiple cores may:

- reorder memory accesses
- observe stale page tables
- observe stale TLB entries

Barriers ensure:

- ordering across cores
- visibility across cores
- correctness of shared memory

Teaching Note

Inner Shareable + DSB ensures SMP correctness.

****

****

9.13 — Barriers and Speculation

****

****

Speculative execution may:

- read stale TTBR values
- use stale descriptors
- fill TLB entries speculatively

Barriers prevent:

- speculative use of stale mappings
- speculative execution across MMU changes

Implementation Note

ISB is required after MMU configuration changes.

****

****

9.14 — Barriers and Hypervisor Interaction

****

****

Hypervisor uses barriers for:

- Stage-2 page table updates
- Stage-2 TLB invalidation
- VM migration
- VM memory remapping
- nested virtualization

Teaching Note

Hypervisor must use DSB + ISB after TLBI.

****

****

9.15 — Barriers and Nested Virtualization

****

****

Nested virtualization requires:

- nested TLBI
- nested BBM
- nested page table updates
- nested barrier sequences

Implementation Note

Nested barriers must be carefully ordered to avoid deadlocks.

****

****

9.16 — Barriers and QNX/RTOS Behavior

****

****

QNX uses barriers for:

- deterministic memory access
- safe page table updates
- partition isolation
- device driver correctness
- SMP scheduling

Teaching Note

RTOS environments require strict ordering guarantees.

****

****

9.17 — Common Barrier Mistakes

****

****

Mistake 1 — Using DMB instead of DSB

DMB does not guarantee completion.

Mistake 2 — Missing ISB after MMU changes

Pipeline may use stale TTBR/TCR.

Mistake 3 — Missing DSB after TLBI

TLB invalidation may not complete.

Mistake 4 — Missing BBM sequence

TLB aliasing may occur.

****

****

9.18 — Full Barrier Sequence Examples

****

****

Example 1 — Page Table Update

```
STR x0, [descriptor]
DSB SY
TLBI VAE1, x1
DSB SY
ISB
```

Example 2 — Changing TTBR

```
MSR TTBR0_EL1, x0
DSB SY
ISB
```

Example 3 — Changing SCTLR_EL1

```
MSR SCTLR_EL1, x0
ISB
```

****

****

9.19 — Chapter Summary

****

****

This chapter covered:

- DSB, DMB, ISB
- ordering rules
- pipeline effects
- page table update requirements
- TLBI sequences
- BBM
- device memory
- DMA
- SMP systems
- speculation
- hypervisor interaction
- nested virtualization
- QNX/RTOS considerations
- common mistakes
- full examples